Assignment No	**9**
Title	**Linked list**
Objective	**Singly Linked list- Deletion, Reverse, Search, Count**
Roll No	**MCA2649**

Source Code:-

```
//Name:- Satish Rathod
//Roll No:- 2649
//LinkedList


//1)Singly Linked list
//Design a Node class
class Node
{
	int data;
	Node next;
	Node(int data)
	{
		this.data=data;
		this.next=null;
	}
}
class SinglyLinkedList
```

```
{
    // define head
    private Node head=null;
    // a method to check is list empty
    public boolean isEmpty()
    {
        return head==null;
    }
    // insert at the begining
    public void insertFirst(int data)
    {
        //creat node
        Node temp = new Node(data);
        //link to old head
        temp.next = head;
        head = temp; // move the head
    }
    //insert at the end
    public void insertLast(int data)
    {
        //create node
        Node temp = new Node(data);
        //check is the list empty
        if(isEmpty())
        {
```

```
                head=temp;
        }
        // traverse the list till end
        Node tr = head; //start traversal
        while(tr.next != null)
        {
                tr = tr.next; // walk till end and stop at end
        }
        tr.next=temp; // assign new node at the end
}
// Display the list
public void display()
{
        Node temp=head;
        while(temp != null)
        {
                //print each node
                System.out.print(temp.data + "=>");
                // move to the next node
                temp = temp.next;
        }
        System.out.println("Null");

}
//Insert at given position
```

```
public void insertAt(int pos,int data)
{
        if(pos == 1)
        {
                //call insert First
                insertFirst(data);
                return;
        }
        // traverse to the specified position
        Node temp = head;
        for(int i=1; i<(pos-1) && (temp!=null); i++)
        {
                temp=temp.next;
                if(temp==null)
                {
                        System.out.println("Invalid position!");
                        return;
                }
        }
        // assign new node
        Node newN = new Node(data);
        newN.next=temp.next; // assign the newnode to old
        temp.next=newN; // assign new node at specified position
}
// A function to count nodes in the linked list
```

```
public int count()
{
        int cnt=0;
        Node temp=head;
        while(temp != null)
        {
                //increment the count
                cnt++;
                temp = temp.next;
        }
        return cnt;
}

// Search a Node
public int search(int key)
{
        int pos=1;
        Node temp = head;
        while(temp != null)
        {
                //check for the key
                if(temp.data == key)
                return pos;
                temp = temp.next;
                pos++;
```

```
        }
        return -1;
    }
    // delete a node
    public boolean delete(int key)
    {
        // 1) If head is null retuen false
        if(head == null)
        return false;
        // 2) if key is head
        if(head.data == key)
        {
            head = head.next; // move the head
            return true;
        }
        // 3) any other node
        Node prev = head;
        // check whether the node exists
        while(prev.next != null && prev.next.data != key)
            prev = prev.next;
        if(prev.next == null) // if not found
        return false;
        prev.next = prev.next.next;
        return true;
    }
```

```
        public void reverse()
        {
                // three variables
                Node curr;
                Node prev = null;
                Node next = null;
                curr = head;
                while(curr != null)
                {
                        next = curr.next;
                        curr.next = prev;
                        prev = curr;
                        curr = next;
                }
                head = prev;
        }

}
class SinglyDemo
{
        public static void main(String args[])
        {
                SinglyLinkedList l1 = new SinglyLinkedList();
                System.out.println("Is Empty:"+l1.isEmpty());
                l1.insertFirst(10);
```

```
l1.insertLast(20);
l1.insertLast(30);
l1.insertLast(40);
System.out.println("Before Insertion");
l1.display();
System.out.println("After Insertion");
l1.insertAt(2,50);
l1.display();
System.out.println("number of Nodes: " +l1.count());
int p = l1.search(20);
if(p>0)
{
        System.out.println("Found at :" + p);
}
else
{
        System.out.println("not Found");
}
if(l1.delete(40))
{
        System.out.println("Deleted");
}
else
{
        System.out.println("Not Found");
```

```
            }
            l1.display();
            l1.reverse();
            System.out.println("Reverse LinkedList");
            l1.display();
        }
}
```

OUTPUT:-

